

Problem Set #7

Due date: November 22, 2024, 11:59 PM

REMINDER: As you work on your coding assignments, it is important to remember that passing the examples provided does not guarantee full credit. While these examples can serve as a helpful starting point, it is ultimately your responsibility to create additional tests, ensure that it is functioning correctly and meets all the requirements and specifications outlined in the assignment instructions. Your grade for this assignment is divided in two parts:

50% > Passing all post-due date tests
(no partial credit for failed cases)

50% > Clarity and design of your code
(Class Coding Standards, requirements/specifications)

Before you start...

- **You are not allowed to use material that has not been covered in class.**
- Your code must be written using Python language.
- Function names must be defined as stated in the function description. Failure to comply with this requirement will result in a 0 score for the visible and post-due date testing.
- No code should be written outside functions, including function calls. All testing code must be done inside the `main` function
- Your functions should NOT have any `input()` calls in its body.
- Your code should be concise and efficient without useless code or redundant code.
- Failure to follow the assignment's requirements will result in a zero score for the assignment even if the autograder gives you full score.
- Code submitted with syntax errors does not receive ANY credit.
- Revise the Grading Notes provided in the Canvas Assignment that includes the evaluation criteria for Clarity and Design.
- You will be writing several functions, but they will all be saved in one file: `PS6.py`. Please save all the functions in this one file or you will lose points.
- Ask questions using our Problem Set 7 channel in Microsoft Teams.

– To receive any credit for any of these problems, you are **not allowed** to use loops in any form, global variables, or materials that we have not covered in class.

⚠ Before you get into code writing, remember these two points:

- Recursion is all about making your parameters smaller: This should manifest in two ways. First, every recursive call you make should take in smaller parameters than the call you're currently in. And second, your base case should reflect the smallest value you ever want your parameters to have. (Don't address your second-smallest or third-smallest parameter in the base case. Only the very smallest one)
- Stay in the same frame: You shouldn't get caught up worrying whether your recursive call works. You just need to trust that it will do what you want. Take the factorial example from lectures. If you're testing out factorial(5), you should only worry about what happens inside that frame. Don't worry how factorial(4) gets handled, just assume it works and it will return 24. If you try to think about what goes on inside the frame for factorial(4), and then factorial(3), and so on, then you've fallen down a rabbit hole you'll never climb out of. Don't think about how the recursive call works. Just think, "What should the recursive call evaluate to?" and assume it will evaluate to that. **Focus only on the current frame, assuming every recursive call outputs what it should.**

You might define helper functions, but those helper functions must:

- Be exactly named as presented in lectures, adding `_helper` to the name of the function
- It must be recursive, and it must follow all directions of the assignment

Part 1: Warm-up

Problem 1.1 (10 pts) Write the Python code to implement the recursive function `skipping` that takes an integer greater or equal to zero and returns an integer that represents the sum of every other integer from the input value to zero (inclusive).

Example:

```
>>> skipping(11) # 11 + 9 + 7 + 5 + 3 + 1 + 0
36
```

Problem 1.2 (10 pts) Write the Python code to implement the recursive function `zig_zag` that takes an integer greater or equal to zero and returns a string of stars (*) to form a zig-zag format. Your return value must include the newline character to separate each row of stars.

Example:

```
>>> zig_zag(3)
"***\n**\n*\n**\n***" # Graded output
>>> print(zig_zag(3)) # Output visualization
***
**
*
*
**
***
```

Problem 1.3 (10 pts) Write the Python code to implement the recursive function `product_of_digits` that takes a positive integer and returns an integer that represents the product of every digit in the given input excluding zeros.

Example:

```
>>> product_of_digits(1024)    # 1 * 2 * 4
8
```

Part 2: We heard you like recursion!

Problem 2.1 (10 pts) Write the Python code to implement the recursive function `near_by_unique` that takes a positive integer and returns an integer where adjacent digits of the same value are removed. Converting the input to other types such as `str` is not allowed. Use floor division (`//`) and modulo (`%`) operations as done in lectures. You cannot use strings, lists, dictionaries or tuples in any way in this problem.

Example:

```
>>> near_by_unique(22224666666782)
246782
```

Problem 2.2 (10 pts) Write the Python code to implement the recursive function `combine_lists` that takes two lists of numbers and a string of size one representing a mathematical operation (`+`, `-`, `*`, `/`). The function returns a new list where each element is the result of applying the given operation to the corresponding elements of both lists. You can assume the input lists have the same length. Your function should not mutate the input lists in any way.

Example:

```
>>> combine_lists([10, 20, 30], [1, 2, 3], '*')
[10, 40, 90]
```

Problem 2.3 (10 pts) Write the Python code to implement the recursive function `to_evens` that takes a list of integers and returns a new list where every odd element in the list is increased by 1. Your function should not mutate in any way the original list

Example:

```
>>> my_list = [-5, 10, 19, 26, 8, 7]
>>> to_evens(my_list)
[-4, 10, 20, 26, 8, 8]
>>> my_list
[-5, 10, 19, 26, 8, 7]
```

Problem 2.4 (10 pts) Write the Python code to implement the recursive function `to_evens_decructive`, which is the destructive version of Problem 2.4. In other words, this version mutates the original list, and does not create new ones.

Example:

```
>>> my_list = [-5, 10, 19, 26, 8, 7]
>>> to_evens_destructive(my_list)
>>> my_list
[-4, 10, 20, 26, 8, 8]
```

Part 3: All about GCD

Problem 3.1 (15 pts) The greatest common divisor (GCD) of two numbers is the largest number that divides them both. For instance, the greatest common factor of 20 and 15 is 5, since 5 divides both 20 and 15 and no larger number has this property. Write the Python code to implement the recursive function `right_cumulative_gcd` that takes a list of positive numbers and returns a new list where each element is replaced by the GCD of itself and all elements to its right. Your function should not mutate the original list.

Example:

```
>>> right_cumulative_gcd([20, 50, 15, 40, 10])
[5, 5, 5, 10, 10]
```